

EX NO: 9

SIMULATION OF CONVOLUTIONAL CODES AND ANALYSIS OF ENCODED OUTPUT BIT SEQUENCES USING MATLAB

DATE:

Objectives

- (i) To implement convolutional encoding for a given binary input sequence using a rate $\frac{1}{2}$ convolutional encoder and tabulate the corresponding encoded output pairs.
- (ii) To visualize and compare the input bit sequence and encoded output bit sequence using MATLAB plots to analyze the redundancy introduced by convolutional coding.

MATLAB code

```
1 clc;
2 clear;
3 close all;
4
5 % Fixed input sequence
6 input_bits = [1 0 0 1 1];
7
8 % Convolutional encoder
9 trellis = poly2trellis(3,[7 5]);
10 encoded_bits = convenc(input_bits,trellis);
11
12 % Separate encoded bits into pairs
13 encoded_pairs = reshape(encoded_bits,2,[]);
14
15 % Create output strings
16 output_pair = cell(length(input_bits),1);
17
18 for i = 1:length(input_bits)
19     output_pair{i} = sprintf('%d%d', ...
20         encoded_pairs(i,1), encoded_pairs(i,2));
21 end
22
23 % Create table
24 Bit_Position = (1:length(input_bits))';
25 Input_Bit = input_bits';
26 Output_Encoded = output_pair;
27
28 T = table(Bit_Position, Input_Bit, Output_Encoded);
29
```

```
1/MATLAB Drive/experiment_9.m
28 T = table(Bit_Position, Input_Bit, Output_Encoded);
29
30 disp('Convolutional Encoding Table');
31 disp(T);
32
33 % Plot
34 figure;
35
36 subplot(2,1,1);
37 stairs(input_bits,'LineWidth',1.5);
38 ylim([-0.2 1.2]);
39 title('Input Bit Sequence');
40 xlabel('Bit Position');
41 ylabel('Bit Value');
42 grid on;
43
44 subplot(2,1,2);
45 stairs(encoded_bits,'LineWidth',1.5);
46 ylim([-0.2 1.2]);
47 title('Encoded Output Bit Sequence');
48 xlabel('Bit Position');
49 ylabel('Bit Value');
50 grid on;
```

Objective ::

To implement convolutional encoding for a given binary input sequence using a rate $\frac{1}{2}$ convolutional encoder and tabulate the corresponding encoded output pairs.

Procedure

1. Open MATLAB and create a new script file.
2. Initialize the MATLAB workspace using `clc`, `clear`, and `close all`.
3. Define the input bit sequence as `[1 0 0 1 1]`.
4. Create a convolutional encoder trellis using a constraint length of 3 and generator polynomials `[7 5]`.
5. Encode the input sequence using the `convenc()` function.
6. Group the encoded bits into pairs corresponding to each input bit.
7. Create and display a table containing the bit position, input bit, and encoded output pair.

8. Observe the encoded output pairs generated by the encoder.

Tabulation

Bit position	Input bit	Encoded sequence
1	1	11
2	0	10
3	0	11
4	1	11
5	1	01

Result

The convolutional encoder was successfully implemented using a constraint length of 3 and generator polynomials [7 5]. The input bit sequence [1 0 0 1 1] was encoded into the output sequence 1110111101, and the corresponding encoded output pairs were tabulated successfully.

Objective (ii): To visualize and compare the input bit sequence and encoded output bit sequence using MATLAB plots to analyze the redundancy introduced by convolutional coding.

Procedure

1. Create a figure window in MATLAB.
2. Plot the input bit sequence using the stairs() function in the first subplot.
3. Label the axes and provide a suitable title.
4. Plot the encoded output bit sequence using the stairs() function in the second subplot.
5. Label the axes and provide a suitable title.
6. Enable the grid for clear visualization.
7. Compare the input and encoded sequences.
8. Observe that the encoded sequence contains twice as many bits as the input sequence due to the rate $1/2$ encoding process.

Model graph

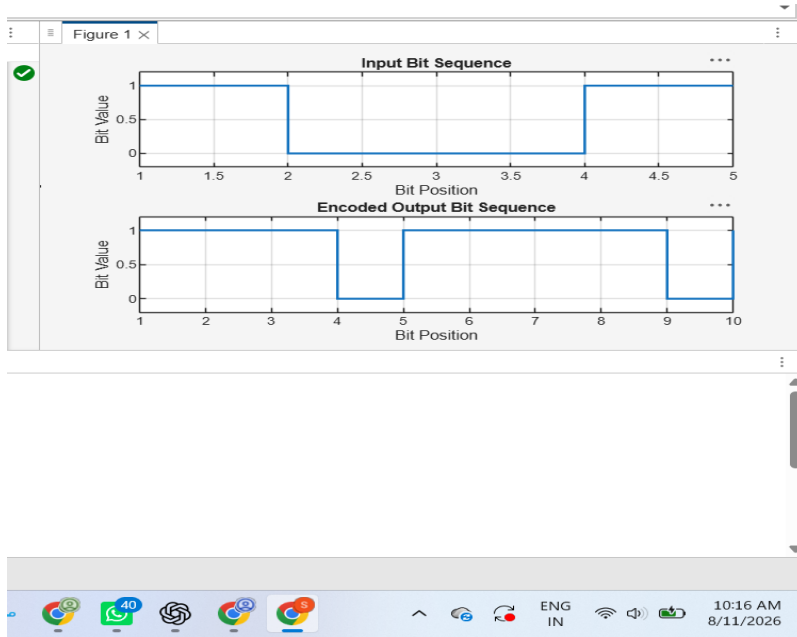


Fig. Input and Encoded sequence

Result:

The input and encoded bit sequences were plotted using MATLAB. The encoded sequence contained 10 bits for 5 input bits, confirming a rate $\frac{1}{2}$ convolutional code. The plots clearly demonstrated the redundancy introduced by convolutional encoding, which enhances the error-correction capability of digital communication systems.